

BickSpec (.bks)

Especificación del lenguaje y estado del proyecto - Fase IV

Fase IV: Plugin de VS Code · Fecha: 19 de Mayo de 2026

Este documento consolida el estado del proyecto BickSpec en su Fase IV, enfocada exclusivamente en el plugin de Visual Studio Code. Retoma el compilador funcional cerrado en Fase III y documenta la capa de integración para edición, ejecución, diagnóstico, empaquetado VSIX y uso permanente dentro de VS Code. No documenta BickSpec Studio ni el IDE desktop; esa línea pertenece a otra capa de producto.

Componente	Estado en Fase IV
Plugin de VS Code	Implementado como extensión instalable mediante VSIX.
Reconocimiento de archivos .bks	Funcional con lenguaje BickSpec asociado a la extensión.
Syntax highlighting	Implementado mediante gramática de resaltado para VS Code.
Snippets y ayudas de edición	Implementados para constructos centrales del lenguaje.
Comandos de ejecución	Run Current File, Run Folder, Output, artefactos y documentación.
Integración con compilador	Uso del JAR bickspec-compiler-1.0.0.jar empaquetado en la extensión.
Diagnósticos	Lectura de errores léxicos, sintácticos y semánticos hacia VS Code.
Programas interactivos	READ ejecutado en terminal integrada para permitir entrada del usuario.
Setup Wizard	Validación de Java, compilador bundled, pruebas y configuración avanzada.
Distribución final	VSIX generable localmente y por GitHub Actions como artifact.

1. Equipo y repositorio

El proyecto mantiene la continuidad de las fases previas y conserva el repositorio principal del lenguaje como fuente del compilador, la gramática y el plugin.

Nombre	Carné
Paolo Marroquin	1146822
Zane Lockridge	1373724
Martin Sierra	1244523

Repositorio del proyecto: <https://github.com/paolomarroq/bickspec-lang>

2. Relación de Fase IV con las fases anteriores

La Fase IV no redefine el lenguaje ni modifica el compilador. Su propósito es convertir el trabajo de las fases anteriores en una experiencia utilizable desde Visual Studio Code. La extensión actúa como una capa de productividad sobre el JAR final construido en Fase III.

- Fase I dejó definida la especificación léxica y la gramática base del lenguaje.
- Fase II consolidó el parser, el visitor inicial, árboles DOT/SVG y traducción dirigida por sintaxis.
- Fase III cerró el ciclo del compilador: semántica, tabla de símbolos, Java generado, build y execution.
- Fase IV integra ese compilador en VS Code mediante una extensión instalable y empaquetable.

Alcance explícito: Fase IV documenta únicamente el plugin de VS Code. BickSpec Studio, instaladores desktop, reportes visuales del IDE y el wizard de la aplicación de escritorio quedan fuera de este documento.

3. Descripción general del plugin de VS Code

El plugin de BickSpec para VS Code permite trabajar con archivos .bks sin abandonar el editor. Su objetivo principal es que un usuario pueda escribir un programa BickSpec, recibir ayuda visual del editor, ejecutar el compilador y revisar resultados o artefactos desde la misma interfaz.

Capacidades principales:

- Asociación automática de archivos con extensión .bks al lenguaje BickSpec.
- Resaltado de sintaxis para keywords, strings, operadores, comentarios, funciones, imports, FX y monedas.
- Snippets para estructuras frecuentes como PROJECT, DISPLAY, READ, IF/ELSE, WHILE, FX y NPV/PAYBACK.
- Comandos desde Command Palette y menús contextuales del editor.
- Ejecución del compilador final desde VS Code sin abrir manualmente terminales complejas.
- Soporte para programas no interactivos y programas interactivos con READ.
- Lectura de salida del compilador para diagnostics, output channel y enlaces a artefactos generados.
- Documentación local empaquetada y setup wizard para validar Java y compilador.

La extensión se diseña para usuarios finales y no solo para desarrollo. Por eso el flujo final reemplaza la ejecución con F5 por instalación permanente mediante VSIX.

4. Objetivos específicos de Fase IV

- Reducir la fricción de uso del compilador fuera del IDE Java o de comandos Maven.
- Permitir que VS Code reconozca BickSpec como lenguaje propio.
- Hacer que Run Current File funcione con el compiler JAR incluido en la extensión.
- Separar el uso final del plugin del modo Extension Development Host.
- Conservar opciones avanzadas para desarrolladores sin exigir Git, Maven o un clone local al usuario final.
- Producir un paquete VSIX verificable, distribuible y regenerable con GitHub Actions.

5. Arquitectura técnica del plugin

La extensión se organiza como una capa ligera alrededor del compilador. El editor contiene los recursos necesarios para reconocer el lenguaje, ejecutar el JAR y exponer comandos al usuario.

Elemento	Responsabilidad
vscode-extension/package.json	Metadatos, comandos, activation events, icono, contribuciones de lenguaje y scripts.
extension.js	Entry point de la extensión, registro de comandos y coordinación de ejecución.
syntaxes/	Gramática de resaltado de sintaxis para archivos .bks.
snippets/	Plantillas de código para acelerar escritura de programas BickSpec.
language-configuration.json	Reglas de comentarios, brackets, auto-close pairs e indentación básica.
media/compiler/	Ubicación del bickspec-compiler-1.0.0.jar bundled para uso final.
media/docs/	Documentación HTML o recursos locales accesibles desde VS Code.
setup/	Vista o servicios del setup wizard para Java, compilador y prueba de entorno.
scripts/prepare-compiler.js	Copia el JAR construido hacia media/compiler antes de empaquetar el VSIX.

Flujo interno simplificado del plugin:

```
Archivo .bks en VS Code
↓
Activación de la extensión BickSpec
↓
Syntax highlighting + snippets + comandos
↓
Resolución del compiler JAR
↓
Ejecución: java -jar bickspec-compiler-1.0.0.jar archivo.bks
↓
Output channel / Terminal integrada / Diagnostics / Artefactos
```

6. Funcionalidades de edición

La primera capa visible del plugin es la experiencia de escritura. El usuario obtiene señales visuales y plantillas para trabajar con la sintaxis real de BickSpec.

6.1 Reconocimiento del lenguaje

Los archivos .bks quedan asociados al lenguaje BickSpec. Esto permite activar resaltado, snippets y comandos contextuales sin configurar manualmente cada archivo.

6.2 Syntax highlighting

El resaltado identifica palabras reservadas, imports, FX, monedas, operadores, funciones built-in, strings y comentarios. Su objetivo no es validar semánticamente el programa, sino facilitar lectura y edición.

6.3 Snippets principales

Snippet / Patrón	Uso esperado
PROJECT	Crear estructura PROJECT "Nombre" { ... }.
DISPLAY	Emitir texto o valores calculados.
READ	Declarar entrada interactiva desde terminal.
IF / ELSE / END	Estructura condicional del lenguaje.
WHILE / DO / END	Ciclo de repetición con condición.
FX	Declarar tipo de cambio fuera del PROJECT.
NPV / PAYBACK	Usar built-ins financieros del runtime.

6.4 Identidad visual del plugin

La extensión conserva la identidad BickSpec basada en Spec Grid: lenguaje de especificación, grilla, brackets, precisión financiera y paleta técnica navy/teal. El paquete VSIX incluye icono oficial y nombre estable para evitar duplicados de instalación.

7. Integración con el compilador

La Fase IV depende del JAR final construido en Fase III. El plugin no reimplementa lexer, parser, semántica ni generación Java; invoca el compilador empaquetado.

Prioridad de resolución del compiler JAR:

- Ruta personalizada elegida por el usuario, si existe y es válida.
- JAR bundled dentro del VSIX: media/compiler/bickspec-compiler-1.0.0.jar.
- Fallback de desarrollo: app/target/bickspec-compiler-1.0.0.jar cuando se ejecuta desde el repositorio.
- Error claro si no existe compilador disponible.

Comando conceptual de ejecución:

```
java -jar <extension>/media/compiler/bickspec-compiler-1.0.0.jar archivo.bks
```

7.1 Ejecución no interactiva

Cuando el programa no contiene READ, la extensión puede ejecutar el compilador y mostrar el resultado en Output Channel o Program Output. Esto cubre archivos como operaciones aritméticas, NPV sin input, pruebas negativas y generación de artefactos.

7.2 Ejecución interactiva con READ

Los programas con READ requieren entrada del usuario. Por eso el plugin usa la terminal integrada de VS Code, permitiendo que el usuario escriba los valores solicitados por el programa generado.

En PowerShell, cuando el ejecutable está entre comillas, se debe anteponer el operador de llamada &. Esta corrección evita errores como Unexpected token -jar en programas interactivos.

```
& "java" -jar "<compilerJar>" "<archivo.bks>"
```

8. Diagnósticos, output y artefactos

El plugin aprovecha la salida estructurada del compilador de Fase III para acercar la experiencia de usuario a la de un lenguaje real dentro del editor.

Salida del compilador	Uso dentro del plugin
[STATUS]	Determina éxito o fallo general de parse, semántica, build y execution.
[SYMBOLS]	Permite abrir o enlazar la tabla de símbolos CSV generada.
[TREE]	Permite localizar árboles DOT/SVG generados por el compilador.
[JAVA]	Permite abrir el archivo Java generado desde VS Code.
[BUILD]	Permite informar si javac generó .class correctamente.
[EXECUTION]	Permite mostrar resultado de ejecución o indicar que el programa espera input.
[ERROR]	Permite construir diagnostics para Problems panel.

8.1 Problems panel

Los errores léxicos, sintácticos y semánticos pueden mapearse a diagnósticos de VS Code. Esto permite ubicar líneas problemáticas y diferenciar categorías como LEX01, SYN01, SEM01, SEM02, SEM03 y SEM04.

8.2 Artefactos navegables

- Java generado en output/java.
- Símbolos exportados a CSV en output/symbols.
- Árboles de parseo DOT/SVG en output/trees.
- Logs o reportes de ejecución cuando el compilador los genera.

El plugin no altera el formato de estos artefactos; solo los hace visibles y accesibles desde el editor.

9. Setup Wizard de la extensión

El setup wizard tiene como objetivo reducir errores comunes de instalación y validar que el entorno puede ejecutar BickSpec sin depender de configuraciones manuales complejas.

Paso	Validación
Java Check	Verifica que java esté disponible y pueda ejecutarse.
Bundled Compiler Check	Comprueba que el JAR incluido en media/compiler exista.
Workspace Check	Valida que exista un archivo o carpeta de trabajo para pruebas.
Setup Test	Ejecuta una prueba básica del compilador desde el plugin.
Ready	Marca la extensión como lista para uso normal.

9.1 Java como requisito externo

El usuario final no necesita Git ni Maven para usar la extensión instalada. El requisito externo esencial es Java, porque el compilador bundled se ejecuta con `java -jar`.

Si Java no está instalado, el wizard debe mostrar instrucciones o acciones para instalar Java 21. En Windows puede ofrecer winget si está disponible; en macOS debe guiar al usuario hacia Adoptium y distinguir entre Apple Silicon e Intel.

9.2 Opciones avanzadas

La extensión conserva rutas avanzadas para desarrolladores: seleccionar un JAR personalizado, usar un repositorio local de bickspec-lang o reconstruir el compilador. Estas opciones no son obligatorias para el uso final.

10. Empaquetado VSIX y distribución

El cierre de Fase IV reemplaza el flujo de desarrollo con F5 por instalación permanente mediante VSIX. El VSIX se comporta como el instalador oficial del plugin de VS Code.

Comandos de generación local:

```
mvn -f app/pom.xml package
cd vscode-extension
npm run prepare:compiler
npm run check
npm run package:vsix
```

Comando de instalación local:

```
code --install-extension ./bickspec-finance-dsl-0.3.0.vsix --force
```

10.1 Contenido esperado del VSIX

Contenido	Propósito
extension.js	Entry point del plugin.
package.json	Metadatos, comandos, contribuciones y activación.
syntaxes/**	Resaltado de sintaxis.
snippets/**	Plantillas de escritura.
language-configuration.json	Configuración de brackets, comentarios e indentación.
media/compiler/bickspec-compiler-1.0.0.jar	Compilador incluido para uso final.
media/docs/**	Documentación local del lenguaje/plugin.
setup/**	Recursos del setup wizard.

10.2 Workflows de GitHub Actions

La distribución no requiere versionar binarios generados como app/target ni .vsix. Los workflows pueden generar el JAR y el VSIX como artifacts, manteniendo el repositorio limpio.

11. Comandos expuestos por la extensión

La extensión presenta comandos claros para que el usuario trabaje con BickSpec desde Command Palette o acciones contextuales.

Comando	Descripción
BickSpec: Run Current File	Ejecuta el archivo .bks activo con el compilador.
BickSpec: Run Folder	Ejecuta o procesa una carpeta de archivos .bks cuando aplica.
BickSpec: Show Compiler Output	Abre el canal de salida del compilador.
BickSpec: Open Generated Java	Abre el Java generado por la ejecución reciente.
BickSpec: Open Symbol Table CSV	Abre la tabla de símbolos exportada.
BickSpec: Open Parse Tree SVG	Abre el árbol de parseo generado si existe.
BickSpec: Open Setup Wizard	Abre el asistente de configuración.
BickSpec: Validate Environment	Valida Java, compilador y configuración.
BickSpec: Select Compiler JAR	Permite elegir un JAR personalizado.
BickSpec: Run Setup Test	Ejecuta una prueba mínima del ambiente.
BickSpec: Reset Setup	Reinicia estado/configuración del wizard.
BickSpec: Open Documentation	Abre la documentación local del plugin/lenguaje.

La estabilidad del identificador del paquete permite reinstalar versiones nuevas con --force sin crear extensiones duplicadas.

12. Suite de pruebas para Fase IV

Además de la suite del compilador, Fase IV requiere pruebas específicas de integración con VS Code. Estas pruebas verifican que la extensión no solo compile, sino que funcione como herramienta permanente del editor.

Prueba	Objetivo
Instalación VSIX	Instalar con code --install-extension y verificar que BickSpec aparezca en VS Code.
Reconocimiento .bks	Abrir un archivo .bks y confirmar language mode BickSpec.
Syntax highlighting	Validar colores para PROJECT, DISPLAY, READ, IF, FX, NPV y strings.
Snippets	Insertar plantillas de PROJECT, IF, READ, FX y NPV/PAYBACK.
Run no interactivo	Ejecutar P1/P2/P6 y revisar Program Output.
Run interactivo	Ejecutar P3 o prueba READ y confirmar terminal integrada.
Diagnósticos léxicos	Abrir prueba con token inválido y verificar LEX01.
Diagnósticos sintácticos	Abrir prueba con expresión incompleta y verificar SYN01.
Diagnósticos semánticos	Usar variable no declarada y verificar SEM01.
Artefactos	Abrir Java generado, símbolos CSV y árbol SVG desde comandos.
Documentación	Abrir documentación local desde Command Palette.
Wizard	Validar Java, compiler bundled y setup test.

12.1 Resultado esperado de una prueba interactiva

```
PROJECT "Input Test" {  
  DISPLAY "Ingrese el valor de A:"  
  READ A  
  DISPLAY "Ingrese el valor de B:"  
  READ B  
  SUMA := A + B  
  DISPLAY "Suma:"  
  DISPLAY SUMA  
}
```

La ejecución debe abrir terminal integrada, solicitar valores y continuar sin errores de PowerShell o rutas.

13. Flujo de uso final

El flujo final de usuario queda diseñado para ser directo y permanente dentro de VS Code.

```
Descargar o generar bickspec-finance-dsl-<version>.vsix
↓
Instalar VSIX en VS Code
↓
Abrir VS Code normalmente
↓
Abrir archivo .bks
↓
Ejecutar BickSpec: Run Current File
↓
Revisar Output / Terminal / Diagnostics / Artefactos
```

El modo F5 queda reservado únicamente para desarrollo interno del plugin. Para usuarios finales o pruebas de entrega se debe usar instalación por VSIX.

14. Estructura final del repositorio para Fase IV

```
bickspec-lang/
  app/
    pom.xml
    src/main/java/...          # compilador de Fase III
  docs/
    BickSpec.g4
    BickSpec_Spec_ANTLR_FaseIV_Plugin_VSCode.pdf
  testing/
    P1_HolaMundo.bks ...
  vscode-extension/
    extension.js
    package.json
    language-configuration.json
    syntaxes/
    snippets/
    setup/
    media/
      compiler/bickspec-compiler-1.0.0.jar
      docs/
      scripts/prepare-compiler.js
  .github/workflows/
    build-compiler.yml
    build-vscode-extension.yml
```

15. Alcance y límites de Fase IV

La Fase IV puede considerarse completa cuando la extensión se instala como VSIX, reconoce archivos .bks, ejecuta el compilador incluido, maneja programas interactivos, muestra diagnostics y permite abrir documentación o artefactos desde VS Code.

Queda fuera del alcance de esta fase:

- BickSpec Studio como aplicación desktop independiente.
- Instaladores EXE/DMG de una aplicación Electron.
- Reportes visuales avanzados del IDE desktop.
- Publicación en Visual Studio Marketplace.
- Firma/notarización de software de escritorio.
- Cambios en gramática, semántica o runtime financiero del compilador.

16. Estado final de la Fase IV

El plugin de VS Code completa la transición de BickSpec desde un compilador ejecutable por terminal hacia una herramienta utilizable por desarrolladores y usuarios técnicos dentro de un editor moderno. La extensión no reemplaza el compilador; lo encapsula y facilita su uso.

- El usuario final puede instalar la extensión sin usar F5.
- El compiler JAR queda incluido dentro del VSIX o generado por workflow antes del empaquetado.
- El flujo normal no exige Git, Maven ni clonar bickspec-lang.
- Java permanece como requisito externo para ejecutar el JAR.
- Los comandos de VS Code permiten correr, diagnosticar y navegar resultados del compilador.
- Los artifacts del compilador se conservan como evidencia técnica de cada ejecución.

Con esto, Fase IV queda cerrada como capa de integración del compilador BickSpec en Visual Studio Code, manteniendo continuidad con Fase I, Fase II y Fase III sin alterar el núcleo del lenguaje.